claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-afea05a2916e454fa.jsonl`
- SHA-256: `c1724be0f0187533b9efc9738042460f3d028e18bd5c7e6081dfb9b4bf83ee0a`
- Source modified: `2026-07-02T03:03:27+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T03:02:21.725Z)

```
You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is
2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/khelsutra
(avidullu/khelsutra):

{
  "title": "Issue #98 still valid: demo rally grid keeps showing while the user's upload
processes",
  "category": "tech-debt",
  "severity": "P2",
  "file": "web/src/App.tsx",
  "line": 75,
  "evidence": "App.tsx:75-77 defaults rallies/order to DEMO_RALLIES whenever no videoId, and the
pick view (App.tsx:349-371) renders ModeBanner('demo') + RallyGrid with those demo rallies
unconditionally; IngestPanel's busy state is internal (IngestPanel.tsx:104) and never lifted to
App, so an in-flight upload/processing job leaves the static demo selection on screen.",
  "proposed_fix": "Lift the ingest busy/phase signal from IngestPanel to App (callback or lifted
hook) and swap the demo grid for a per-job 'your reel is processing' state that becomes the real
picker on done. Minimal version (hide demo grid + show processing banner when busy) is S; the
full acceptance in the issue is M. Pairs naturally with #99 and #111 as one processing-UX PR
series.",
  "effort": "M",
  "tracked_in": "khelsutra#98"
}

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer,
C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-
data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator,
C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field;
non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup',
C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).
VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited
code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent
commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete
failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it
owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,
product/strategy/licensing, real cloud spend, counsel ToS/DPA?
HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only
commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you
personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,
say REFUTED; if a commit fixed it, ALREADY_FIXED.
```

2. user (2026-07-02T03:02:30.684Z)

```
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)
master
```

3. user (2026-07-02T03:02:31.359Z)

```
1       import { useEffect, useMemo, useState } from "react";
2       import { useTranslation, Trans } from "react-i18next";
3       import { useRallySelection } from "./hooks/useRallySelection";
4       import { RallyGrid } from "./components/RallyGrid";
5       import { QueryBar } from "./components/QueryBar";
6       import { TapFilters } from "./components/TapFilters";
7       import { SelectionFooter } from "./components/SelectionFooter";
8       import { LanguageSwitcher } from "./components/LanguageSwitcher";
9       import { CompileResult, type BuildState } from "./components/CompileResult";
10      import { IngestPanel } from "./components/IngestPanel";
11      import { FfmpegBanner } from "./components/FfmpegBanner";
12      import { VideoLibrary } from "./components/VideoLibrary";
13      import { AnnotatePanel } from "./components/AnnotatePanel";
14      import { SetupWizard, isWizardDone } from "./components/SetupWizard";
15      import { runQuery, type ParsedQuery } from "./lib/query";
16      import { floorRisk } from "./lib/floor";
17      import { reelFilename } from "./lib/format";
18      import { classifyError } from "./lib/errors";
19      import { errorText } from "./lib/errorText";
20      import { log, newCorrelationId } from "./lib/log";
21      import { ApiError, compileReel, getConfig, highlightUrl, listRallies, listVideos,
minShotCount } from "./api/client";
22      import { videoId as videoIdOf, videoName } from "./lib/video";
23      import type { EngineConfig, RallySegment } from "./api/types";
24
25      /** One compact, structured description of how a query was understood (for the trace
log). */
26      function parseTrace(q: ParsedQuery) {
27        return {
28          raw: q.raw,
29          recognized: q.recognized,
30          filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
31          sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
32          limit: q.limit ?? null,
33          unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
34        };
35      }
36
37      // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
38      // contract; only honest fields are surfaced. A real session loads from POST /api/query.
39      const DEMO_RALLIES: RallySegment[] = [
40        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
41        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
42        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
43        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
44        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
45        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
46      ];
47
```

```
48      type LoadState =
49        | { status: "demo" }
50        | { status: "loading" }
51        | { status: "loaded"; count: number }
52        | { status: "error"; error: string };
53
54      /** Annotate-in-UI (P8) is behind a runtime feature flag ? default OFF (D13: ships gated
until consented
55       *  / no-minors footage is in hand). Turn it on per-browser with `?annotate=1` or
56       *  `localStorage['khelsutra.annotate'] = '1'`. No build step; the owner enables it when
ready. */
57      function annotateFlag(): boolean {
58        try {
59          if (new URLSearchParams(window.location.search).get("annotate") === "1") return
true;
60          return localStorage.getItem("khelsutra.annotate") === "1";
61        } catch {
62          return false;
63        }
64      }
65
66      export default function App() {
67        const { t, i18n } = useTranslation();
68        // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
69        // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
70        // without a reload (it also rewrites ?video= so a refresh is stable).
71        const [videoId, setVideoId] = useState<string | null>(
72          () => new URLSearchParams(window.location.search).get("video"),
73        );
74
75        const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
76        const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
77        const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
78        const [reelName, setReelName] = useState("highlights");
79        const [build, setBuild] = useState<BuildState>({ status: "idle" });
80        // P2c: the first-run wizard shows once per browser, before the ingest panel, when
nothing's loaded.
81        const [wizardDone, setWizardDone] = useState(isWizardDone);
82        // The human name of the loaded video ? shown PERSISTENTLY so the operator always
knows which match
83        // they're editing. Once a video loads, the library + ingest panel (which showed the
name) unmount,
84        // so without this the name would vanish. Set from the library on click; else resolved
below.
85        const [videoLabel, setVideoLabel] = useState<string | null>(null);
86        // P8 (flagged): switch the loaded-video view between picking rallies and annotating
them.
87        const [annotateEnabled] = useState(annotateFlag);
88        const [view, setView] = useState<"pick" | "annotate">("pick");
89
90        // Display order is separate from selection: a query can re-sort the cards while the
shared
91        // selection set (sel) stays the single source of truth for what lands in the reel.
92        const sel = useRallySelection(rallies, "all");
93
94        // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
95        useEffect(() => {
96          if (!videoId) return;
97          const cid = newCorrelationId();
98          log("info", "rallies.load_start", { cid, video_id: videoId });
99          // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
```

```
100         // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
101         setLoadState({ status: "loading" });
102         let alive = true;
103         listRallies(videoId, cid)
104           .then((rs) => {
105             if (!alive) return;
106             setRallies(rs);
107             setOrder(rs);
108             sel.apply(rs.map((r) => r.id));
109             setLoadState({ status: "loaded", count: rs.length });
110             log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
111           })
112           .catch((err) => {
113             if (!alive) return;
114             // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
115             // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
116             const e = classifyError(err);
117             setLoadState({ status: "error", error: errorText(t, e) });
118             log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
119           });
120         return () => {
121           alive = false;
122         };
123         // videoId is stable for the page; sel.apply is a stable callback.
124         // eslint-disable-next-line react-hooks/exhaustive-deps
125       }, [videoId]);
126
127       // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
128       // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
129       const [cfg, setCfg] = useState<EngineConfig | null>(null);
130       useEffect(() => {
131         let alive = true;
132         getConfig()
133           .then((c) => alive && setCfg(c))
134           .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
135         return () => {
136           alive = false;
137         };
138       }, []);
139       const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
140
141       // Stable chronological ordinal (by start_time) so re-sorting never renumbers a rally.
142       const ordinalOf = useMemo(() => {
143         const m = new Map<number, number>();
144         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
145         return (id: number) => m.get(id) ?? 0;
146       }, [rallies]);
147
148       // Load a video into the picker ? from a fresh ingest (IngestPanel) OR reopening a
stored one
149       // (VideoLibrary, which passes the match name). Rewrites ?video= so a refresh is
stable (D10).
150       const loadVideo = (id: string, label?: string) => {
151         setVideoId(id);
152         setVideoLabel(label ?? null);
153         window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
154       };
155
156       // Resolve the loaded video's name when we don't already have it (a deep-link ?video=
```

```
on reload, or
157       // an ingest hand-off) so the current-video label is always populated ? best-effort,
from the same
158       // GET /api/videos the library uses. A miss (not yet listed) keeps the friendly
fallback.
159       useEffect(() => {
160         if (!videoId || videoLabel) return;
161         let alive = true;
162         listVideos()
163           .then((vs) => {
164             const v = vs.find((x) => videoIdOf(x) === videoId);
165             const name = v ? videoName(v) : "";
166             if (alive && name) setVideoLabel(name);
167           })
168           .catch((err) => log("warn", "video.name_resolve_failed", { video_id: videoId,
error: String(err) }));
169         return () => {
170           alive = false;
171         };
172       }, [videoId, videoLabel]);
173
174       // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
175       const onBuild = async () => {
176         if (build.status === "building") return;
177         const cid = newCorrelationId();
178         const filename = reelFilename(reelName);
179         const ids = [...sel.selected];
180         if (!videoId) {
181           log("info", "build.no_video", { cid, selected: ids.length });
182           setBuild({
183             status: "error",
184             error: t("build.noVideo"),
185           });
186           return;
187         }
188         setBuild({ status: "building" });
189         log("info", "build.start", {
190           cid,
191           video_id: videoId,
192           filename,
193           selected: ids.length,
194           total_sec: Math.round(sel.totalSec),
195           below_floor: warning?.atRisk.length ?? 0,
196         });
197         try {
198           // Pass the active UI locale so the engine localizes the reel's branding watermark
(LW1).
199           const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage
?? i18n.language);
200           const url = highlightUrl(videoId, filename);
201           setBuild({ status: "done", url, filename });
202           log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
203         } catch (err) {
204           // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
205           // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
206           const status = err instanceof ApiError ? err.status : undefined;
207           const e = classifyError(err);
208           setBuild({ status: "error", error: errorText(t, e) });
209           log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
210         }
```

```
211       };
212
213       // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
214       // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
215       // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
216       const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
217         const cid = newCorrelationId();
218         log("info", "query.parse", { cid, source, ...parseTrace(q) });
219
220         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
221         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
222         if (q.unavailable.length > 0) {
223           log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
224         }
225
226         if (!q.recognized) {
227           log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
228           return; // empty / unparseable ? leave the user's curation untouched
229         }
230
231         try {
232           const { ordered, selectedIds } = runQuery(rallies, q);
233           setOrder(ordered);
234           sel.apply(selectedIds);
235           log("info", "query.apply", {
236             cid,
237             source,
238             filters: q.filters.length,
239             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
240             limit: q.limit ?? null,
241             total: rallies.length,
242             selected: selectedIds.length,
243             selected_ids: selectedIds,
244             dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
245           });
246           if (selectedIds.length === 0) {
247             // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
248             log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
249           }
250         } catch (err) {
251           // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
252           log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
253         }
254       };
255
256       // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
257       const onToggle = (id: number) => {
258         log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
259         sel.toggle(id);
260       };
261       const onBulk = (op: "select_all" | "clear" | "invert") => {
262         const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
263         log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
```

```
        if (op === "select_all") sel.selectAll();
        else if (op === "clear") sel.clearAll();
        else sel.invert();
      };

      return (
        <main className="min-h-screen bg-paper text-ink pb-24">
          <header className="bg-ink text-white">
            <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
              <span className="text-xl font-extrabold tracking-tight">
                Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
              </span>
              <LanguageSwitcher />
            </div>
          </header>

          <section className="mx-auto max-w-5xl px-6 py-10">
            {/* Hard-blocker warning: FFmpeg absent ? indexing + reel compile can't run.
Persistent (unlike
                the once-per-browser wizard), shown only when the engine positively reports
it missing. */}
            <FfmpegBanner />
            <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
              {t("app.badge")}
            </span>
            <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
            <p className="mt-2 max-w-2xl text-ink/70">
              <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
            </p>

            {/* Persistent "current video" label ? always visible once a video is loaded, so
the operator
                never loses track of which match they're editing (the library/ingest panel
that showed the
                name has unmounted by now). Name from the library, else resolved from GET
/api/videos. */}
            {videoId && (
              <p
                className="mt-3 inline-flex items-center gap-2 rounded-full bg-ink/5 px-3
py-1 text-sm text-ink/70"
                data-testid="current-video"
              >
                <span aria-hidden="true">?</span>
                <Trans
                  i18nKey="app.currentVideo"
                  values={{ name: videoLabel ?? t("app.currentVideoUnknown") }}
                  components={{ b: <strong className="text-ink" /> }}
                />
              </p>
            )}

            {/* P8 (flagged): switch the loaded video between picking rallies and annotating
them. */}
            {videoId && annotateEnabled && (
              <div className="mt-4 inline-flex rounded-full border border-black/10 p-1 text-
sm" data-testid="view-toggle">
                <button
                  type="button"
                  onClick={() => setView("pick")}
                  aria-pressed={view === "pick"}
                  className={"min-h-11 rounded-full px-4 " + (view === "pick" ? "bg-ink
```

```
text-white" : "text-ink/70")}
                >
                  {t("annotate.tabPick")}
                </button>
                <button
                  type="button"
                  onClick={() => setView("annotate")}
                  aria-pressed={view === "annotate"}
                  className={"min-h-11 rounded-full px-4 " + (view === "annotate" ? "bg-ink
text-white" : "text-ink/70")}
                >
                  {t("annotate.tab")}
                </button>
              </div>
            )}

            {/* P2c first-run wizard: before anything is loaded, walk a new user through AI-
key / compute /
                videos (from /api/capabilities). Shows once per browser; then the ingest
panel takes over. */}
            {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}

            {/* Front-half (P6 + B-P2): when no video is loaded, offer BOTH ? reopen a
stored video from
                "Your videos" (VideoLibrary ? GET /api/videos), or add a new one
(IngestPanel ? process ?
                load). Once a video_id exists, the existing back-half (pick ? build) takes
over. */}
            {!videoId && wizardDone && (
              <>
                <VideoLibrary onSelect={loadVideo} />
                <IngestPanel onLoaded={loadVideo} />
              </>
            )}

            {/* P8 annotate view (flagged): the labeler over the loaded video. */}
            {view === "annotate" && videoId && <AnnotatePanel key={videoId}
videoId={videoId} />}

            {/* Pick-rallies view (the default + the whole app when annotate is off). */}
            {view === "pick" && (
              <>
                <ModeBanner load={loadState} videoId={videoId} />

                <div className="mt-6">
                  <QueryBar onApply={onApplyQuery} />
                  {/* A9: localized tap controls for users who can't type the English query
syntax. */}
                  <TapFilters onApply={onApplyQuery} />
                </div>

                <div className="mt-6 flex flex-wrap gap-2 text-sm">
                  <button type="button" onClick={() => onBulk("select_all")}
className="inline-flex min-h-11 items-center rounded-full border border-black/15 px-4
hover:border-green">{t("bulk.selectAll")}</button>
                  <button type="button" onClick={() => onBulk("clear")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
                  <button type="button" onClick={() => onBulk("invert")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
                </div>

                <div className="mt-5">
                  <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
```

```
ordinalOf={ordinalOf} />
367                 </div>
368
369                 <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })}
/>
370               </>
371             )}
372           </section>
373
374           {/* The reel-building footer belongs to the pick view only (annotation has its own
save). */}
375           {view === "pick" && (
376             <SelectionFooter
377               count={sel.count}
378               totalSec={sel.totalSec}
379               reelName={reelName}
380               onReelNameChange={setReelName}
381               onBuild={onBuild}
382               building={build.status === "building"}
383               warning={warning}
384             />
385           )}
386         </main>
387       );
388     }
389
390     function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
391       if (load.status === "demo") {
392         return (
393           <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
394             <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
395           </p>
396         );
397       }
398       if (load.status === "loading") {
399         return (
400           <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
401             <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
}} />
402           </p>
403         );
404       }
405       if (load.status === "error") {
406         return (
407           <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
408             <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
409           </p>
410         );
411       }
412       return (
413         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
414           <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
415         </p>
416       );
417     }
418
```

4. assistant (2026-07-02T03:02:38.583Z)

Now let me check IngestPanel's busy state and the issue status.

5. user (2026-07-02T03:02:39.529Z)

```
1       // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2       // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3       // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4       // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5       // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6       // (done but no rallies) is shown, never silently treated as a load.
7       import { useEffect, useState } from "react";
8       import type { ChangeEvent, FormEvent } from "react";
9       import { useTranslation } from "react-i18next";
10      import { createAsset, getCapabilities } from "../api/client";
11      import type { StorageBackendInfo } from "../api/types";
12      import { useIngestJob } from "../hooks/useIngestJob";
13      import { useUpload } from "../hooks/useUpload";
14      import { classifyError, type IngestError } from "../lib/errors";
15      import { errorText } from "../lib/errorText";
16      import { log, newCorrelationId } from "../lib/log";
17      import { StatusChip, Spinner } from "./StatusChip";
18
19      type Compute = "local" | "cloud";
20      // The three storage-medium choices the picker offers. "bucket" covers s3 OR gcs (the
user pastes a
21      // full scheme'd URI); "drive" is a mounted Google Drive folder
(STUDIO_MEDIA_LIFECYCLE.md P5, D14).
22      type Medium = "local" | "bucket" | "drive";
23
24      interface Props {
25        /** Called with the engine's video_id once a submitted job finishes successfully. */
26        onLoaded: (videoId: string) => void;
27      }
28
29      export function IngestPanel({ onLoaded }: Props) {
30        const { t, i18n } = useTranslation();
31        const [uri, setUri] = useState("");
32        const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
33        // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
34        const locale = i18n.resolvedLanguage ?? i18n.language;
35
36        // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
37        // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
38        // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before it
can run.
39        const [cloudAvailable, setCloudAvailable] = useState(false);
40        const [compute, setCompute] = useState<Compute>("local");
41        const [cloudAck, setCloudAck] = useState(false);
42        // Storage-medium picker (P5): which media THIS box can store into
(capabilities.storage.backends,
43        // usable-only). Absent/local-only ? no picker (today's flow unchanged).
44        const [backends, setBackends] = useState<StorageBackendInfo[]>([]);
45        const [medium, setMedium] = useState<Medium>("local");
46        const [mediumUri, setMediumUri] = useState("");
47        const [storing, setStoring] = useState(false);
48        const [storeError, setStoreError] = useState<IngestError | null>(null);
49        useEffect(() => {
50          let alive = true;
51          getCapabilities()
52            .then((c) => {
```

```
            if (!alive) return;
            setCloudAvailable(c.deployment?.cloud_available === true);
            setBackends(c.storage?.backends ?? []);
          })
          .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
        return () => {
          alive = false;
        };
      }, []);
      // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
      const sendCompute = cloudAvailable ? compute : undefined;
      // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
      // don't dispatch a paid job without consent).
      const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;

      // Which medium segments to offer: local always; "bucket" if s3 OR gcs is usable;
"drive" if a Drive
      // mount is usable. The picker only appears when there's a REAL choice (a non-local
usable backend).
      const canUse = (id: string) => backends.some((b) => b.id === id && b.usable);
      const canBucket = canUse("s3") || canUse("gcs");
      const canDrive = canUse("gdrive");
      const mediumOptions: Medium[] = ["local", ...(canBucket ? ["bucket" as const] : []),
...(canDrive ? ["drive" as const] : [])];
      const showMediumPicker = mediumOptions.length > 1;
      const needsMediumUri = medium !== "local";
      // A non-local medium needs a destination URI before anything can be stored/submitted.
      const mediumBlocked = needsMediumUri && mediumUri.trim() === "";

      // Store the obtained source (an uploaded local path, or a pasted at-rest URI) INTO
the chosen medium,
      // then process the durable copy. A failed store is loud (surfaced like every other
ingest error).
      const storeThenProcess = async (src: { uploadedPath?: string; sourceUri?: string }) =>
{
        const cid = newCorrelationId();
        setStoreError(null);
        setStoring(true);
        try {
          const asset = await createAsset({ ...src, mediumUri: mediumUri.trim() }, cid);
          log("info", "asset.stored", { cid, video_id: asset.video_id, medium: asset.medium,
copied: asset.copied });
          submit(asset.stored_uri, locale, sendCompute); // index the stored copy
        } catch (err) {
          setStoreError(classifyError(err));
          log("error", "asset.store_failed", { cid, error: String(err) });
        } finally {
          setStoring(false);
        }
      };

      // On upload completion: local medium ? process the upload directly (today's flow); a
non-local
      // medium ? store a durable copy into it first, then process. The arrow is recreated
each render
      // (useUpload reads onCompleteRef.current), so `medium`/`mediumUri`/`sendCompute` are
always fresh.
      const upload = useUpload((path) => {
        if (medium === "local") submit(path, locale, sendCompute);
        else void storeThenProcess({ uploadedPath: path });
      });
      const busy = ingestBusy || upload.busy || storing;
      // Disable the submit surfaces while busy, awaiting the cloud cost ack, or missing a
```

```
medium URI.
106       const inputsDisabled = busy || cloudBlocked || mediumBlocked;
107
108       const onSubmitUri = (e: FormEvent) => {
109         e.preventDefault();
110         if (inputsDisabled) return;
111         // The URI form is the SOURCE only in local mode (a bucket URL / host path to index
directly). For
112         // a non-local medium the source is the upload; the text box below is the
DESTINATION, not this.
113         submit(uri, locale, sendCompute);
114       };
115       const onFile = (e: ChangeEvent<HTMLInputElement>) => {
116         const file = e.target.files?.[0];
117         // Symmetric with onSubmitUri: never start while blocked (the disabled input already
prevents this).
118         if (file && !inputsDisabled) upload.start(file);
119         e.target.value = ""; // let re-picking the same file fire change again
120       };
121       const cancelAll = () => {
122         upload.cancel();
123         cancelIngest();
124       };
125       const resetAll = () => {
126         upload.reset();
127         resetIngest();
128         setStoreError(null);
129       };
130
131       // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
132       // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
133       // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
134       const errorState =
135         upload.phase.status === "error"
136           ? upload.phase.error
137           : storeError ?? (phase.status === "error" ? phase.error : null);
138       const errorMsg = errorState ? errorText(t, errorState) : null;
139
140       return (
141         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
142           <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
143           <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
144
145           {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud
GPU. Default
146               LOCAL; CLOUD reveals a cost note + a required acknowledgement before any
submit can run. */}
147           {cloudAvailable && (
148             <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
149               <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
150                 {t("ingest.compute.legend")}
151               </legend>
152               <div className="mt-1 flex flex-wrap gap-2">
153                 {(["local", "cloud"] as const).map((opt) => (
154                   <button
155                     key={opt}
156                     type="button"
157                     aria-pressed={compute === opt}
158                     disabled={busy}
```

```
159                     onClick={() => setCompute(opt)}
160                     className={
161                       "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
162                       (compute === opt
163                         ? "border-green bg-green/10 text-ink"
164                         : "border-black/15 text-ink/70 hover:border-green")
165                     }
166                   >
167                     {t(`ingest.compute.${opt}`)}
168                   </button>
169                 ))}
170               </div>
171               {compute === "cloud" && (
172                 <div className="mt-3" data-testid="compute-cost">
173                   <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
174                   <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
175                     <input
176                       type="checkbox"
177                       checked={cloudAck}
178                       disabled={busy}
179                       onChange={(e) => setCloudAck(e.target.checked)}
180                       className="h-4 w-4"
181                     />
182                     {t("ingest.compute.ack")}
183                   </label>
184                   {cloudBlocked && (
185                     <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
186                       {t("ingest.compute.ackRequired")}
187                     </p>
188                   )}
189                 </div>
190               )}
191             </fieldset>
192           )}
193
194           {/* MEDIUM PICKER (P5, D14): where to STORE this video. Rendered only from the
media this box can
195               actually use (capabilities.storage.backends, usable-only), and only when
there's a real
196               choice beyond local. A non-local medium reveals a destination-URI field; the
video is stored
197               there (a durable, MD5-keyed copy ? free-space checked) and then indexed. Self-
host posture. */}
198           {showMediumPicker && (
199             <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="medium-picker">
200               <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
201                 {t("ingest.medium.legend")}
202               </legend>
203               <div className="mt-1 flex flex-wrap gap-2">
204                 {mediumOptions.map((opt) => (
205                   <button
206                     key={opt}
207                     type="button"
208                     aria-pressed={medium === opt}
209                     disabled={busy}
210                     onClick={() => {
211                       setMedium(opt);
212                       setStoreError(null);
213                     }}
214                     data-testid={`medium-${opt}`}
215                     className={
```

```
                      "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
                      (medium === opt
                        ? "border-green bg-green/10 text-ink"
                        : "border-black/15 text-ink/70 hover:border-green")
                    }
                  >
                    {t(`storage.${opt}`)}
                  </button>
                ))}
              </div>
              {needsMediumUri && (
                <div className="mt-3" data-testid="medium-dest">
                  <input
                    type="text"
                    value={mediumUri}
                    onChange={(e) => setMediumUri(e.target.value)}
                    placeholder={medium === "drive" ? t("ingest.medium.drivePlaceholder") :
t("ingest.medium.bucketPlaceholder")}
                    aria-label={t("ingest.medium.destAria")}
                    disabled={busy}
                    className="min-h-11 w-full max-w-md rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
                  />
                  <p className="mt-2 text-xs text-ink/50">{t("ingest.medium.destHint")}</p>
                  {mediumBlocked && (
                    <p className="mt-1 text-xs text-amber-700" data-testid="medium-
required">
                      {t("ingest.medium.destRequired")}
                    </p>
                  )}
                </div>
              )}
            </fieldset>
          )}

          {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
          <div className="mt-4">
            <label
              className={
                // focus-within makes the visible button show a focus ring when the (sr-
only) input is
                // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
                "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
                (inputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
              }
            >
              {t("ingest.chooseFile")}
              <input
                type="file"
                accept=".mp4,.mov,video/mp4,video/quicktime"
                aria-label={t("ingest.uploadAria")}
                disabled={inputsDisabled}
                onChange={onFile}
                className="sr-only"
              />
            </label>
            <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
          </div>

          {/* SECONDARY: in LOCAL mode, point at a cloud bucket / host path to index
directly. Hidden for a
```

```
              non-local medium, where the destination field above owns the URI box and the
upload is the
              source (avoids two conflicting URI inputs). */}
            {medium === "local" && (
            <div className="mt-4 border-t border-black/5 pt-4">
              <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
              <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
                <input
                  type="text"
                  value={uri}
                  onChange={(e) => setUri(e.target.value)}
                  placeholder={t("ingest.uriPlaceholder")}
                  aria-label={t("ingest.uriAriaLabel")}
                  disabled={inputsDisabled}
                  className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
                />
                <button
                  type="submit"
                  disabled={inputsDisabled}
                  className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
                >
                  {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
                </button>
              </form>
              <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
            </div>
            )}

            {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
                (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
            <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
              {upload.phase.status === "uploading" && (
                <div data-testid="upload-progress">
                  {/* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
                      % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/}
                  <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
                  <div className="mt-2 flex max-w-sm items-center gap-2">
                    <div
                      className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
                      role="progressbar"
                      aria-label={t("ingest.uploadProgress")}
                      aria-valuenow={upload.phase.pct}
                      aria-valuemin={0}
                      aria-valuemax={100}
                    >
                      <div className="h-full bg-green transition-all" style={{ width:
`${upload.phase.pct}%` }} />
                    </div>
                    <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
                      {upload.phase.pct}%
                    </span>
                  </div>
                  <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
                  <button
                    type="button"
                    onClick={cancelAll}
```

```
326                   className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
327                 >
328                   {t("ingest.cancel")}
329                 </button>
330               </div>
331             )}
332             {storing && (
333               <div data-testid="ingest-storing">
334                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.medium.storing")} />
335               </div>
336             )}
337             {ingestBusy && (
338               <div>
339                 <StatusChip
340                   tone="info"
341                   icon={<Spinner />}
342                   label={
343                     t("ingest.working") +
344                     (phase.status === "polling" && phase.progress
345                       ? ` · ${t("ingest.progress", { progress: phase.progress })}`
346                       : "")
347                   }
348                 />
349                 {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
350                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
351                 <button
352                   type="button"
353                   onClick={cancelAll}
354                   className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
355                 >
356                   {t("ingest.cancel")}
357                 </button>
358               </div>
359             )}
360             {errorMsg && (
361               <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
362                 <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
363                 <p className="mt-2">{errorMsg}</p>
364                 <button
365                   type="button"
366                   onClick={resetAll}
367                   className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
368                 >
369                   {t("ingest.retry")}
370                 </button>
371               </div>
372             )}
373           </div>
374         </section>
375       );
376     }
377
```

6. user (2026-07-02T03:02:41.127Z)

```
f9eea14 web: resilient video-summary readers + show fps in the library (#140)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
cc06c32 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged) (#128)
9e9df72 feat(web): keep the video name visible after open (persistent current-video label)
```

```
(#127)
39c1349 feat(web): P6 ? "Your videos" library (reopen a stored video) (#124)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
(#88)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
46e0244 feat(web): A9 ? localized tap-only filter controls (#68)
501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*)
(#62)
95195a5 feat(web): A4 ? mobile touch targets (?44px) + visible tooltip text (#61)
4525c5d feat(web): B-P2 ? in-UI bucket ingest (point at a gs:// video ? process ? load) (#56)
289324e feat(web): extract all SPA strings to the en catalog + i18n parity CI ? tracker B2 (#45)
5b95b0d feat(web): i18n scaffold + language switcher (en-only) ? tracker B1 (#42)
155153d feat(web): lead with rally length; park shot-count as experimental (owner) (#33)
8bf200d feat(web): per-rally MVP P5 ? load real rallies + Build?compile?preview?download (MVP
complete) (#31)
---
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
f67dfe7 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b) (#104)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
ca619fb feat(web): T4 ? in-UI file upload (the runnable-tool keystone) (#66)
92a6c45 feat(web): A8 ? background-processing reassurance + icon+text status chips (#63)
501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*)
(#62)
95195a5 feat(web): A4 ? mobile touch targets (?44px) + visible tooltip text (#61)
4525c5d feat(web): B-P2 ? in-UI bucket ingest (point at a gs:// video ? process ? load) (#56)
```

7. assistant (2026-07-02T03:02:50.552Z)

```
The code matches the claim. Let me verify the issue status on GitHub.
```

8. user (2026-07-02T03:02:55.250Z)

```
{"labels":[],"n_comments":0,"state":"OPEN","title":"SPA: replace the demo rally-selection the
moment an upload/processing starts (so the user knows their reel is
coming)","url":"https://github.com/avidullu/khelsutra/issues/98"}
---
{"state":"OPEN","title":"SPA: keep the uploaded video's filename visible during processing (it
disappears mid-job)"}
{"state":"OPEN","title":"Resilient processing UX: survive page refresh + continually-updating
progress"}
```

9. assistant (2026-07-02T03:03:14.047Z)

```
All checks complete. The claim verifies against the current working tree (master @ f9eea14) and
the issue is still open upstream.
```

10. user (2026-07-02T03:03:27.239Z)

```
Structured output provided successfully
```